

מסמך אפיון מאוחד ומלא — מנגנון ההמלצות

מסמך בנייה עצמאי — כולל כל מה שנדרש למימוש מאפס, ללא תלות במסמכים קודמים

גרסה 1.1 (מאוחד) | 14 באוגוסט 2026

מסמך זה מאחד ומחליף את שני המסמכים הקודמים (RecommendationArchitecture_Spec v0.8 ו-ContactRecommendationBatch_Spec v1.0) לכדי מפרט אחד, שלם ועצמאי. הוא כולל: מודל נתונים מלא (סכימת כל הטבלאות), את כל אלגוריתמי הניקוד, שני זרימות ההמלצה המלאות (עצמית ולאיש קשר, כולל pagination לולאת הפידבק, שכבת שיתוף אירועים/קבוצות/רשימות תפוצה (פרק 14), קונפיגורציה מומלצת לכל קבוע שניתן לכיול, ורשימת endpoints מרומזת. נקודות שעדיין דורשות החלטת מוצר (לא טכנית) מסומנות במפורש ומרוכזות בנספח א' בסוף המסמך, יחד עם ערך ברירת מחדל מומלץ לכל אחת — כך שאפשר לממש הכל מיידית ולכייל מאוחר יותר.

תוכן עניינים

1. סקירה כללית

1.1 הבעיה שהמנגנון פותר

1.2 גישת הפתרון

1.3 שני מנגנוני הצריכה

2. מודל הנתונים המלא

2.1 user_profiles

2.2 contacts

2.3 events

2.4 self_gift_suggestions

2.5 recommendations

2.6 good_gifts_catalog

2.7 Master Tag List

2.8 gift_stats_<tag> (12 טבלאות)

2.9 gift_country_stats

2.10 gift_neighbors

2.11 gift_history (לא ב-MVP)

2.12 recommendation_calls, api_usage

2.13 כל האינדקסים הנדרשים

2.14 hosted_events

2.15 groups

2.16 group_members

2.17 invite_links

2.18 event_audience

3. קביעת הפרופיל האפקטיבי (עבור המנגנון לאיש קשר)

3.1 מקושר מול לא-מקושר

3.2 פתרון סתירות בתחומי עניין

3.3 גיל, מדינה ודת

4. ליבת החישוב המשותפת

4.1 סניטציה

4.2 סינון גס (SQL)

4.3 שליפת סטטיסטיקת הסגמנטים

4.4 primaryTag ו-ageBucket

4.5 Backoff היררכי דו-ממדי

4.6 Bayesian Smoothing

4.7 רשימת המועמדים המדורגת

4.8 קרבה דמוגרפית

4.9 סינון שיתופי מבוסס פריטים (CF)

5. זרימה — המלצות עצמיות

5.1 Trigger וזרימה כללית

5.2 הרכב ה-15 batch (פריטים)

6. זרימה — המלצות לאיש קשר

6.1 בקשה ראשונה

6.2 "חפש עוד" — Compute-Pagination מבוסס-Compute

6.3 תקרה ומעקב session

7. לולאת הפידבק

7.1 פידבק במנגנון העצמי (1-5 + סיבה)

7.2 מיפוי סיבה לפעולה

7.3 פידבק במנגנון לאיש קשר (בינארי)

7.4 מנגנון "צ'אנס שני"

7.5 יעד הכתיבה של פידבק

7.6 category_tag

8. בניית הפרומפט וקריאה ל-Gemini

9. Rate Limiting

10. Pre-processing — נתוני זרע

11. Edge Cases

12. תכונות שנדחו במפורש (לא ב-MVP)

13. רשימת API Endpoints מרומזת

14. שיתוף אירועים, קבוצות ורשימות תפוצה

14.1 hosted_events מול events — שני מושגים שונים

14.2 חברות בקבוצה — הזמנה ישירה מול קישור

14.3 פתרון קהל היעד של אירוע

14.4 כלל הנגישות (Reachability)

נספח א' — קבועים לכיול וסיכום החלטות מוצר פתוחות

1.סקירה כללית

1.1הבעיה שהמנגנון פותר

Giftly הוא אפליקציית המלצות מתנה. הגישה הנאיבית — קריאה חד-פעמית ל- LLM (Gemini) בכל בקשה, ללא זיכרון — אינה משתפרת עם הזמן: דירוגי משתמשים נשמרים אך אינם משפיעים על המלצות עתידיות, לא עבור אותו משתמש ולא עבור משתמשים אחרים עם טעם דומה. המנגנון המתואר במסמך זה בונה שכבת למידה מעל הגישה הזו.

1.2גישת הפתרון

שילוב בין (Exploration חיפוש חופשי של Gemini, לגילוי רעיונות חדשים) לבין Exploitation של קטלוג מתנות פנימי (good_gifts_catalog שנבנה בהדרגה מתוך מתנות שהוכיחו את עצמן. כל מתנה בקטלוג צוברת סטטיסטיקת הצלחה (shown/liked) במבחן רמות גרנדלריות — מהספציפית ביותר (תחום עניין + גיל, או תחום עניין + מדינה) ועד הכללית ביותר (ממוצע גלובלי של אותה מתנה) — ובורר ביניהן באמצעות שרשרת backoff היררכית, מנוקד בעזרת הסתברות מוחלקת (Bayesian smoothing) שמונעת הטיה ממוצרים עם מעט מדי דאטה. שלושה מקורות ניקוד נוספים, אורתוגונליים זה לזה, מוזנים לאותה תשתית: התאמת תחום עניין (הליבה), קרבה דמוגרפית (גיל/מגדר/מדינה, בלי קשר לתחום עניין), וסינון שיתופי מבוסס-פריטים (מה שאנשים עם דירוגים דומים לשלך אהבו).

המערכת גם מבחינה בין מקורות מידע שונים על טעמו של אדם (מה שהוא אמר על עצמו לעומת מה שאחר העריך לגביו — פרק 3), ובין סוגי כישלון שונים בהמלצה (פרק 7) — כדי שכל דירוג יזין בחזרה בדיוק את החלק הנכון של המערכת.

1.3שני מנגנוני הצריכה

אותה ליבת חישוב (פרק 4) מזינה שני מנגנוני צריכה שונים מהותית באופיים, כי מטרתם שונה:

המלצות עצמיות (self)	המלצות לאיש קשר (contact)	
המשתמש הרשום עצמו	אדם אחר (עם או בלי פרופיל Giftly)	למי
אותו אדם שמבקש	צד שלישי מבקש עבור אדם אחר	מי "צורך" את הפלט
גילוי מבוקר (exploration)	ביטחון מקסימלי ("האם זה באמת יתאים")	מטרה עיקרית
קבוע — 15 פריטים (5.2)	דינמי, עד תקרה — 30 פריטים (6.3)	גודל batch
סקאלת 1–5 + שאלת סיבה בדירוג נמוך (7.1)	בינארי: מתאים / לא מתאים, ללא שאלת סיבה (7.3)	פידבק
self_gift_suggestions	recommendations	טבלת יעד

שתי הבהרות עקרוניות: (א) "ליבה משותפת" מתייחסת אך ורק למנוע הניקוד (backoff פרק 4) — לא להרכב ה- batch, שמפורט בנפרד לכל מנגנון (פרקים 5–6). (ב) Negative, תמיד עוקף, Positive, ברמת מקור הסמכות הנכון בלבד — ראו פרק 3.

2.מודל הנתונים המלא

2.1 user_profiles

פרופיל משתמש רשום. משמש כמקור לזרימה העצמית, וגם כמקור "אמת" כשמשתמש אחר יוצר אותו כאיש קשר (linked_user_id).

שדה	Type	משמעות
id	UUID (PK)	מזהה משתמש
display_name	TEXT	שם תצוגה
gender	TEXT	מגדר
birth_date	DATE	לחישוב גיל / (4.4) age_bucket
country	TEXT	ברירת מחדל "IL"
religion	TEXT	אופציונלי — אינו משמש לניקוד כלל, ראו 3.3
interests	TEXT[]	תגיות מה- (2.7) Master Tag List
negative_prefs	TEXT[]	תגיות שנפסלו, נלמד מפידבק על הזרימה העצמית
free_text	TEXT	ניואנסים בטקסט חופשי

שדה	Type	משמעות
bio	TEXT	ביוגרפיה
require_approval_for_group_invites	BOOLEAN DEFAULT false	הגדרת פרטיות אישית — ראו 2.16, 14

2.2 contacts

איש קשר שמשמש שמר לצורך המלצות מתנה. לא חייב להיות משתמש רשום.

שדה	Type	משמעות
id	UUID (PK)	מזהה
user_id	UUID (FK →user_profiles)	מי יצר את איש הקשר (המדרג / מבקש ההמלצה)
linked_user_id	UUID (FK →user_profiles, nullable)	אם קיים — פרופיל Gifty אמיתי של האדם
name, gender, relationship, relationship_status, has_children, religion, country	TEXT	פרטי בסיס
birth_date	DATE, nullable	מועדך ע"י היוצר, לחישוב — age_bucket רלוונטי בעיקר כשאין linked_user_id (3.3)
interests	TEXT[]	לפי הערכת היוצר — רלוונטי בעיקר כשאין linked_user_id, fallback-כ
negative_prefs	TEXT[]	תגיות שהיוצר סימן כשליליות — לוקאלי אצל היוצר בלבד
free_text	TEXT	ניואנסים חופשיים

2.3 events

שדה	Type	משמעות
id	UUID (PK)	מזהה
contact_id	UUID (FK)	לאיזה איש קשר
type	TEXT	סוג (יום הולדת, יום נישואין...)
date, date_type	DATE / TEXT	תאריך
budget_min, budget_max	NUMERIC	טווח תקציב
reminder_days	INT	תזכורת

2.4 self_gift_suggestions

פלט מנגנון ההמלצות העצמיות.

שדה	Type	משמעות
id	UUID (PK)	מזהה
user_id	UUID (FK →user_profiles)	למי ההמלצה
title, description, estimated_price, category, search_query	TEXT / NUMERIC	פרטי ההמלצה
gift_id	UUID (FK →good_gifts_catalog, nullable)	אם ההמלצה הגיעה מהקטלוג הפנימי
category_tag	TEXT, nullable	תגית יחידה מה- Master Tag List לשימוש ב-negative_prefs — WRONG_CONCEPT ראו 7.2, 7.6

שדה	Type	משמעות
batch_id	UUID	מזהה ריצה — כל הצעות שנוצרו יחד מקבלות אותו UI batch_id. מציג רק את האחרון; GROUP BY לניתוח
rating	INT (1–5, nullable)	דירוג המשתמש
feedback_reason	TEXT ENUM (nullable)	— WRONG_CONCEPT WRONG_PRODUCT TOO_GENERIC ראו פרק 7

2.5 recommendations

פלט מנגנון ההמלצות לאיש קשר.

שדה	Type	משמעות
id	UUID (PK)	מזהה
user_id	UUID (FK)	מי ביקש את ההמלצה
contact_id	UUID (FK)	עבור מי
event_id	UUID (FK, nullable)	לאיזה אירוע
gift_id	UUID (FK → good_gifts_catalog, nullable)	אם ההמלצה תואמת מוצר מהקטלוג
title, description, estimated_price, category, search_query	TEXT / NUMERIC	פרטי המתנה שהוצעה
category_tag	TEXT, nullable	תגית יחידה מה- Master Tag List ראו 7.2, 7.6
score	NUMERIC, nullable	הציון המדורג שחושב בזמן השליפה (null עבור המלצות Gemini טהורות ללא match לקטלוג)
rating	INT (1–5, nullable)	בזרימה זו: 5 = 'מתאים', 2 = 'לא מתאים' בלבד — ראו 7.3
feedback_reason	TEXT ENUM (nullable)	תמיד NULL בזרימה זו — אין שלב שאלת-סיבה, ראו 7.3
batch_id	UUID	מזהה — session משותף לכל הפריטים שהוצעו בבקשה אחת, כולל כל עמודי "חפש עוד" — ראו 6.3
source	TEXT ENUM: 'gemini' 'compute'	האם השורה הגיעה מקריאת (Gemini) רק העמוד הראשון או משכבת ה-compute (כל שאר העמודים) — ראו 6.2, 9
created_at	TIMESTAMP	לצורך דדוף לפי חלון זמן ולצורך מנגנון "צ'ינס שני" (7.4)

2.6 good_gifts_catalog

קטלוג מתנות פנימי שנצבר מכלל המשתמשים (שני המנגנונים כאחד). מחזיק רק את פרטי הפריט והמונים הגלובליים — סטטיסטיקות הסגמנטים (תחום עניין/גיל/מגדר/מדינה) יושבת בטבלאות ייעודיות (2.8–2.9).

שדה	Type	משמעות
id	UUID (PK)	מזהה
title, description, estimated_price, category, search_query	TEXT / NUMERIC	פרטי המוצר
tags	TEXT[] (GIN index)	תגיות מה- Master Tag List (2.7)
global_shown, global_liked	INT	מונים גולמיים על פני כל האוכלוסייה — משמשים גם כרמת הבסיס בשרשרת ה-backoff (4.5) את מקור "טופ-קטלוג גלובלי" ישירות, בלי JOIN
last_verified_at	TIMESTAMP	בדיקת תוקף מחיר/זמינות
is_seed	BOOLEAN	האם הוזהר (כן) - seed data (פרק 10)
created_at	TIMESTAMP	

2.7 Master Tag List — רשימת התגיות הסגורה

ה-vocabulary הסגור שמשמש בכל מקום שבו נדרשת "תגית מרשימה סגורה": user_profiles.interests, contacts.interests, good_gifts_catalog.tags, category_tag, ובנוסף "כללי" — לא תחום עניין נבחר, אלא bucket-קלט למתנות שלא נופלות תחת אף תחום ספציפי (למשל בישום, פרחים).

#	תגית	#	תגית
1	ספורט	7	אקסטרים
2	נגינה	8	סדנאות/זוגיות
3	הופעות/סטנדאפ	9	טכנולוגיה/גאדג'טים
4	ציור/אומנות	10	ספרים/ידע
5	קפה/קולינריה	11	גיימינג
6	טיולים/טבע	—	כללי (bucket, לא נבחר ע"י משתמש)

"האזנה למוזיקה" הוצאה בכוונה מהרשימה — נפוצה מדי ולא ספציפית מספיק כדי לשמש אות לבחירת מתנה (בניגוד ל"נגינה", שמעיד על צורך קונקרטי בציד/כלים).

12 — gift_stats_<tag> 2.8 טבלאות סטטיסטיקה לפי תגית

טבלה פיזית נפרדת לכל אחת מ-11 התגיות ואחת לכללי (12 סה"כ: ..., gift_stats_music, gift_stats_sports, gift_stats_general), gift_stats_כולן באותה סכימה. מתנה עם ריבוי תגיות מקבלת שורה נפרדת בכל טבלת תגית רלוונטית — לא שכול פרטי המתנה עצמם (אלה רק ב-, good_gifts_catalog) שורת מונים קלה.

```
CREATE TABLE gift_stats_<tag> (  
    gift_id UUID REFERENCES good_gifts_catalog(id),  
    overall_shown INT DEFAULT 0, overall_liked INT DEFAULT 0, --  
    age_18_24_shown INT DEFAULT 0, age_18_24_liked INT DEFAULT 0,  
    age_25_34_shown INT DEFAULT 0, age_25_34_liked INT DEFAULT 0,  
    age_35_44_shown INT DEFAULT 0, age_35_44_liked INT DEFAULT 0,  
    age_45_54_shown INT DEFAULT 0, age_45_54_liked INT DEFAULT 0,  
    age_55_64_shown INT DEFAULT 0, age_55_64_liked INT DEFAULT 0,  
    age_65p_shown INT DEFAULT 0, age_65p_liked INT DEFAULT 0,  
    gender_male_shown INT DEFAULT 0, gender_male_liked INT DEFAULT 0,  
    gender_female_shown INT DEFAULT 0, gender_female_liked INT DEFAULT 0,  
    PRIMARY KEY (gift_id)  
);
```

גיל ומגדר מקבלים עמודות רחבות כי הדומיין שלהם קטן וקבוע (6 סלי גיל, מגדרים סגורים) — קריא ומהיר, בלי JOIN נוסף. מדינה, לעומת זאת, היא דומיין פתוח וגדל — עמודה רחבה לכל מדינה הייתה יוצרת טבלה ברוחב לא-מבוקר, לכן נשארת בטבלה צרה משותפת (2.9).

2.9 gift_country_stats

```
CREATE TABLE gift_country_stats (  
    gift_id UUID REFERENCES good_gifts_catalog(id),  
    tag TEXT, -- אחת מ-11 התגיות או 'general'  
    country TEXT,  
    shown INT DEFAULT 0, liked INT DEFAULT 0,  
    PRIMARY KEY (gift_id, tag, country)  
);
```

2.10 gift_neighbors

שכנות מתנה-למתנה, מחושבות ב-batch job מתוזמן (4.9) לצורך סינון שיתופי מבוסס-פריטים.

שדה	Type	משמעות
gift_id	UUID (FK —good_gifts_catalog)	המתנה
neighbor_gift_id	UUID (FK —good_gifts_catalog)	מתנה "שכנה" — נוטה להיות מדורגת יחד
similarity	NUMERIC	ציון Jaccard, 0–1
computed_at	TIMESTAMP	מתי ה-batch job האחרון עדכן את השורה

gift_history (2.11 עתידי — לא ב-MVP)

עתידי: לא ממומש ב-gift_history. MVP מתנות שבאמת ניתנו בפועל, ground truth מדירוג-על-הצעה ומנגנון "קניתי בפועל" נדחו לגרסה עתידית. שום מנגנון פעיל במסמך זה תלוי בטבלה הזו. מתועדת כאן לשמירת הקשר בלבד: id (PK), contact_id (FK), title, given_at (DATE), recommendation_id (FK, nullable).

2.12 recommendation_calls, api_usage

טבלאות טכניות ל-5/ per-user recommendation_calls — rate limiting: recommendation_calls (40/ ; global api_usage — Gemini). ראו פרק 9.

2.13 כלל האינדקסים הנדרשים

טבלה	עמודות	סוג	משמש עבור
good_gifts_catalog	tags	GIN	tags & cleanInterests — סינון (4.2)
good_gifts_catalog	estimated_price	B-tree	סינון (4.2) budget_min/budget_max
(<tag> gift_stats (12)	gift_id	B-tree (PK)	שלפה מרוכזת לפי gift_id - Fine Scoring (4.3)
gift_country_stats	gift_id, tag, country	B-tree מורכב, (PK)	שלפה לפי (4.3, 4.8) gift_id+tag+country
gift_neighbors	gift_id	B-tree	שלפת שכנות בזמן בקשה (4.9)
self_gift_suggestions	user_id, rating	B-tree מורכב (	מתנות שדורגו +4 ע"י המשתמש עצמו (6.1)
self_gift_suggestions	user_id, batch_id	B-tree מורכב (	הצגת ה-batch האחרון ב- UI, GROUP BY לניתוח
recommendations	contact_id, created_at	B-tree מורכב (	דופ, חלון 30 יום (6.2)
recommendations	batch_id	B-tree	ספירת פריטים ב-session, שלפת "מה כבר הוצג" (6.3)
recommendations	contact_id, rating, created_at	B-tree מורכב (	איתור מתנות ללא דירוג שעברו X ימים (7.4)
events	contact_id	B-tree	שלפת אירועים קרובים
recommendation_calls	user_id, created_at	B-tree מורכב (	rate limiting per-user (9)
api_usage	date	B-tree	rate limiting global (9)
hosted_events	owner_user_id	B-tree	שלפת האירועים שמשתמש מארח (14)
groups	owner_user_id	B-tree	שלפת הקבוצות שמשתמש יצר (14)
group_members	group_id, user_id	B-tree מורכב, (PK)	רשימת חברי קבוצה; בדיקת חברות (14)
group_members	user_id, status	B-tree מורכב (	"באילו קבוצות אני חבר" — לצורך פתרון קהל היעד (14)
invite_links	token	B-tree ייחודי (	שלפה מהירה בעת לחיצה על קישור הזמנה (14)
event_audience	hosted_event_id	B-tree	פתרון קהל היעד של אירוע (14)
event_audience	target_group_id	B-tree	"אילו אירועים משותפים לקבוצה הזו" (14)
event_audience	target_contact_id	B-tree	"אילו אירועים משותפים לאיש קשר הזה" (14)

2.14 hosted_events

אירוע שמשתמש מארח/חוגג בעצמו (למשל "החתונה שלי") — שונה עקרונית מ-events (2.3) שהיא תזכורת פרטית של מישהו אחר לגבי תאריך של איש קשר. hosted_events הוא ההפך: המשתמש הוא הנחגג, ובוחר למי לחשוף את זה (14).

שדה	Type	משמעות
id	UUID (PK)	מזהה
owner_user_id	UUID (FK —user_profiles)	מי מארח/חוגג את האירוע
type	TEXT	סוג (חתונה, יום הולדת...)
date, date_type	DATE / TEXT	תאריך
description	TEXT	פרטים חופשיים

שדה	Type	משמעות
created_at	TIMESTAMP	

2.15 groups

שדה	Type	משמעות
id	UUID (PK)	מזהה
owner_user_id	UUID (FK →user_profiles)	יוצר הקבוצה — היחיד שיכול לאשר/להזמין (14)
name	TEXT	שם הקבוצה
created_at	TIMESTAMP	

2.16 group_members

אין קשר ישיר בין חברות בקבוצה לבין — (2.2) contacts שתי מערכות נתונים נפרדות לגמרי. חברות בקבוצה דורשת חשבון רשום; אין אפשרות לצרף מישהו שאין לו חשבון.

שדה	Type	משמעות
id	UUID (PK)	מזהה
group_id	UUID (FK →groups)	לאיזו קבוצה
user_id	UUID (FK →user_profiles)	מי — תמיד משתמש רשום, לא contact
status	TEXT ENUM: INVITED MEMBER DECLINED	ראו 14 — תלוי בהגדרת הפרטיות של user_id (require_approval_for_group_invites, 2.1)
joined_via	TEXT ENUM: DIRECT_INVITE LINK	האם הצטרף דרך הזמנה ישירה של היוצר, או דרך קישור שיתוף (2.17)
initiated_by	UUID (FK →user_profiles, nullable)	מי הזמין (null מעבור הצטרפות דרך קישור)
created_at, responded_at	TIMESTAMP	responded_at כש-status מעובר מ-INVITED

2.17 invite_links

טבלה אחידה לשני סוגי קישורי הזמנה: הצטרפות לקבוצה מסוימת, או הוספה לרשימת אנשי הקשר של בעל הקישור ("רשימת התכונה שלי").

שדה	Type	משמעות
id	UUID (PK)	מזהה
token	TEXT (UNIQUE)	המזהה בקישור עצמו, למשל <token>/giftly.app/join/
owner_user_id	UUID (FK →user_profiles)	בעל הקישור
target_type	TEXT ENUM: GROUP CONTACT_LIST	מה קורה בלחיצה — ראו 14
group_id	UUID (FK →groups, nullable)	מאוכלס רק כש-'GROUP'=target_type
created_at	TIMESTAMP	

2.18 event_audience

קהל היעד של — hosted_event אוסף שורות, כל אחת מטרה אחת; הקהל הסופי הוא איחוד (Union של כל השורות, לא בחירה בודדת מביניהן).

שדה	Type	משמעות
id	UUID (PK)	מזהה

שדה	Type	משמעות
hosted_event_id	UUID (FK → hosted_events)	לאיזה אירוע
target_type	TEXT ENUM: GROUP CONTACT ALL_CONTACTS	סוג המטרה — ראו 14
target_group_id	UUID (FK → groups, nullable)	מאוכלס רק כש- target_type='GROUP'
target_contact_id	UUID (FK → contacts, nullable)	מאוכלס רק כש- target_type='CONTACT'; null עבור

3. קביעת הפרופיל האפקטיבי (עבור המנגנון לאיש קשר)

הצעד הראשון והקריטי במנגנון לאיש קשר, לפני כל חישוב ניקוד. אינו רלוונטי למנגנון העצמי — שם אין צד שלישי, ו- effectivePositive/effectiveNegative הם פשוט selfProfile.interests/negative_prefs.

```
const contact = await getContact(contactId);
const selfProfile = contact.linked_user_id
  ? await getUserProfile(contact.linked_user_id)
  : null;
const contactAssessment = { interests: contact.interests, negative_prefs: contact.negative_prefs };
```

3.1 מקושר מול לא-מקושר

איש קשר לא מקושר	איש קשר מקושר (linked_user_id)	מקור interests/negative_prefs
רק contacts	גם self (user_profiles) וגם contacts (הערכת היוצר)	דיוק
תלוי כולו בהערכת היוצר	self מדויק יותר, third-party	לוגיקת פתרון סתירות
לא נדרשת — מקור יחיד	נדרשת (3.2)	

3.2 פתרון סתירות בתחומי עניין: Priority לפי מקור סמכות (לא Union)

עיקרון: אם דני (בפרופיל שלו) סימן תגית כחיובית, אבל יוסי (שיצר את דני כאיש קשר) סימן אותה כשלילית אצל דני — דני מנצח, כי הוא היחיד שבאמת יודע מה הוא אוהב. תקף גם בכיוון ההפוך.

```
function resolveEffectivePreferences(selfProfile, contactAssessment) {
  const selfPositive = new Set(selfProfile?.interests ?? []);
  const selfNegative = new Set(selfProfile?.negative_prefs ?? []);
  const thirdPartyPositive = new Set(contactAssessment.interests ?? []);
  const thirdPartyNegative = new Set(contactAssessment.negative_prefs ?? []);

  const effectivePositive = new Set();
  const effectiveNegative = new Set();
  const allTags = new Set([...selfPositive, ...selfNegative, ...thirdPartyPositive, ...thirdPartyNegative]);

  for (const tag of allTags) {
    if (selfPositive.has(tag) || selfNegative.has(tag)) {
      // חושב השלישי הצד מה משנה לא, מכריעה היא - עצמית עמדה יש
      if (selfPositive.has(tag)) effectivePositive.add(tag);
      else effectiveNegative.add(tag);
    } else {
      // השלישי הצד להערכת אחרת נופלים - עצמית עמדה אין
      if (thirdPartyPositive.has(tag)) effectivePositive.add(tag);
      else if (thirdPartyNegative.has(tag)) effectiveNegative.add(tag);
    }
  }
  return { effectivePositive: [...effectivePositive], effectiveNegative: [...effectiveNegative] };
}
```

- תגית עם עמדה עצמית סותרת להערכת הצד השלישי → העמדה העצמית גוברת, לכל כיוון.

- תגית ללא עמדה עצמית כלל → נופלים ל- fallback.contactAssessment
- איש קשר לא-מקושר — selfProfile הוא null, תגיות עוברות אוטומטית ל- fallback: effectivePositive/effectiveNegative = מה שהיוצר סימן.

3.3 גיל, מדינה ודת — כללי טיפול שונים מתחומי עניין

תחומי עניין הם דעה סובייקטיבית ולכן דורשים לוגיקת פתרון סתירות (3.2). גיל ומדינה הם עובדה כמעט-אובייקטיבית — אין "מי גובר", יש רק ערך אחד אמיתי ושאלה של מאיזה מקור לשלוח אותו:

```
function resolveEffectiveDemographics(selfProfile, contact) {
  return {
    birthDate: selfProfile?.birth_date ?? contact.birth_date ?? null,
    country: selfProfile?.country ?? contact.country ?? null,
  };
}
```

birthDate האפקטיבי מומר ל- (4.4) age_bucket ומשמש בצירוף הזוגי הראשון בשרשרת ה- country (4.5); backoff האפקטיבי משמש בצירוף הזוגי השני.

religion איננו חלק משרשרת ה- backoff ואינו משפיע על ניקוד המתנות כלל, אף שהוא נאסף ונשמר: שימוש בדת כדי לדרג "מה טוב" קרוב מדי להכללה סטריאוטיפית, בעוד ששימוש בגיל/מדינה קרוב יותר להתאמת זמינות ותרבות צריכה. אם בעתיד יעלה צורך אמיתי (למשל סינון קשיח של מוצרים לא-מתאימים דתית), זהו שימוש שונה לגמרי — פילטר בינארי, לא ממד ניקוד.

4. ליבת החישוב המשותפת

החישוב שמריץ שני המנגנונים כאחד, אחרי שפרק 3 קבע (במנגנון לאיש קשר) את effectivePositive / effectiveNegative / effectiveBirthDate / effectiveCountry.

4.1 סניטציה

```
const cleanInterests = effectivePositive.filter(t => !effectiveNegative.includes(t));
const promptNegatives = effectiveNegative.slice(-4); // רק 4 נכנסות האחרונות (token budget)
```

4.2 סינון גס (SQL)

```
SELECT * FROM good_gifts_catalog
WHERE estimated_price BETWEEN :budget_min AND :budget_max
AND tags && :cleanInterests
LIMIT 100;
```

- — tags && cleanInterests ואופרטור overlap של PostgreSQL; חפיפה של תגית אחת לפחות.
- מקרה קצה: 0 תוצאות → מדלגים ישירות ל- exploration (Gemini), בלי Fine Scoring.
- גיל/מדינה אינם חלק מהסינון הגס — הם משפיעים רק על איזו סטטיסטיקה סומכים עליה לכל מועמד (4.3–4.5), לא על אילו מועמדים בכלל רלוונטיים.

4.3 שליפת סטטיסטיקת הסגמנטים (fetchSegmentStats)

כל שאר הפונקציות בפרק זה מצפות לאובייקט gift.segment_stats בצורה { interests, interest_age, interest_country }. האובייקט הזה אינו עמודה מאוחסנת — הוא מורכב בזיכרון משליפה מרוכזת אחת, מיד אחרי הסינון הגס (4.2):

```
async function fetchSegmentStats(candidates) {
  const byTag = groupBy(candidates, c => c.tags); // תגיות קבוצות בכמה להופיע יכולה מתנה
  const result = new Map(); // gift_id -> { interests, interest_age, interest_country }
  for (const [tag, gifts] of byTag) {
    const ids = gifts.map(g => g.id);
    const rows = await db.query(`SELECT * FROM gift_stats_${tag} WHERE gift_id = ANY($1)`, [ids]);
    const countryRows = await db.query(
      `SELECT * FROM gift_country_stats WHERE tag = $1 AND gift_id = ANY($2)`, [tag, ids]
    );
    mergeIntoResult(result, tag, rows, countryRows); // מפתחות "tag|ageBucket" / "tag|country"
  }
}
```

```

return result;
}

```

ה-JOIN מתבצע פעם אחת לכל תגית שמופיעה בקבוצת המועמדים (עד 100 אחרי 4.2), לא פעם לכל מתנה — כדי לא להפוך ל-N+1 queries.

ageBucket-14.4 primaryTag

```

function getPrimaryTag(gift, cleanInterests) {
  const overlap = gift.tags.filter(t => cleanInterests.includes(t));
  if (overlap.length === 0) return null;
  if (overlap.length === 1) return overlap[0];
  // חפיפה מרובה – בוחרים את התגית עם ה-shown הגבוה ביותר בסגמנט
  return overlap.reduce((best, tag) => {
    const shown = gift.segment_stats?.interests?.[tag]?.shown ?? 0;
    const bestShown = gift.segment_stats?.interests?.[best]?.shown ?? 0;
    return shown > bestShown ? tag : best;
  }, overlap[0]);
}

```

בכוונה: כשיש חפיפה מרובה (למשל coffee וrunning), נבחרת רק תגית אחת — לא ממוצע/חיתוך בין תגיות לא-קשורות. תחומי עניין שונים אינם רומזים דמיון בין אנשים שיש להם את שניהם.

```

const AGE_BUCKETS = [
  { max: 24, label: "18-24" }, { max: 34, label: "25-34" }, { max: 44, label: "35-44" },
  { max: 54, label: "45-54" }, { max: 64, label: "55-64" }, { max: Infinity, label: "65+" },
];
function getAgeBucket(birthDate) {
  if (!birthDate) return null;
  const age = calculateAge(birthDate);
  return AGE_BUCKETS.find(b => age <= b.max).label;
}

```

אם birthDate לא ידוע, ageBucket הוא null ויצירוף interest_age פשוט לא נבדק — שרשרת ה-(4.5) backoff ממשיכה לצירוף הבא.

4.5 Backoff היררכי דו-ממדי

שרשרת סגמנטים מהעשיר ביותר לפשוט ביותר; הרמה העשירה הראשונה עם מספיק תמיכה סטטיסטית זוכה. הרעיון: לחפש קודם מה עבד ל"אנשים הכי דומים" (אותו תחום עניין + אותו טווח גיל, או + אותה מדינה), ורק אם אין מספיק דאטה — להסתפק בדמיון גס יותר.

```

function getEffectiveRate(gift, primaryTag, ageBucket, country, thresholds = { pair: 20, tag: 15 }) {
  {
    const pairCandidates = [];
    if (ageBucket) {
      const ageKey = primaryTag + "|" + ageBucket;
      const ageStats = gift.segment_stats?.interest_age?.[ageKey];
      if (ageStats && ageStats.shown >= thresholds.pair) {
        pairCandidates.push({ liked: ageStats.liked, shown: ageStats.shown, source: "interest+age:" + ageKey });
      }
    }
    if (country) {
      const countryKey = primaryTag + "|" + country;
      const countryStats = gift.segment_stats?.interest_country?.[countryKey];
      if (countryStats && countryStats.shown >= thresholds.pair) {
        pairCandidates.push({ liked: countryStats.liked, shown: countryStats.shown, source: "interest+country:" + countryKey });
      }
    }
    if (pairCandidates.length > 0) {
      return pairCandidates.sort((a, b) => b.shown - a.shown)[0]; // עדיפות לזה עם shown יותר
    }
    const tagStats = gift.segment_stats?.interests?.[primaryTag];
    if (tagStats && tagStats.shown >= thresholds.tag) {
      return { liked: tagStats.liked, shown: tagStats.shown, source: "interest:" + primaryTag };
    }
  }
}

```

```

}
return { liked: gift.global_liked, shown: gift.global_shown, source: "global" };
}

```

רמה	נבדק מתי	סף (MIN)	הערה
1. interest+age / interest+country	תמיד, ראשון בתור	MIN_PAIR_SUPPORT = 20	שני הצירופים נבדקים במקביל; אם שניהם עוברים סף, נבחר זה עם ה-shown הגבוה יותר
2. interest בלבד	רק אם אף צירוף זוגי לא עבר סף	MIN_TAG_SUPPORT = 15	רמת ביניים
3. global (של אותה מתנה)	רק אם גם רמה 2 לא עברה סף	—	global_shown/liked של המתנה הספציפית — לא global של כל הקטלוג

שני קבועי הסף נבדלים בכוונה: צירוף זוגי דליל יותר מתג בודד, ולכן דורש רף גבוה יותר לפני שסומכים עליו — אחרת חוזרת בעיית "מתנת נישא" (פרק 11), רק ברמת דיוק גבוהה יותר.

4.6 Bayesian Smoothing

```

function smoothedRate(liked, shown, globalMean, k) {
  const alpha = globalMean * k;
  const beta = (1 - globalMean) * k;
  return (liked + alpha) / (shown + alpha + beta);
}

```

- — GLOBAL_MEAN ה-rate הממוצע על פני כל הקטלוג; ברירת מחדל מומלצת: 0.6.
- — K_PRIOR משקל האמון" בממוצע הגלובלי; ברירת מחדל מומלצת: 10 ברמת tag, 15 ברמת pair (ברמת יותר, נמשך חזק יותר לממוצע).

מתנה	shown	liked	rate גולמי	smoothed_rate (GLOBAL_MEAN=0.6, K=10)
חדשה לגמרי	0	0	—	$(0+6)/(0+10) = 0.60$
מעט דאטה, מושלם	3	3	1.00	$(3+6)/(3+10) = 0.69$
הרבה דאטה, טוב	200	180	0.90	$(180+6)/(200+10) = 0.89$
מעט דאטה, גרוע	5	0	0.00	$(0+6)/(5+10) = 0.40$

4.7 רשימת המועמדים המדורגת

```

const scoredCandidates = candidates
  .map(gift => ({ ...gift, score: computeScore(gift, cleanInterests) }))
  .sort((a, b) => b.score - a.score);
// computeScore -> getPrimaryTag -> getEffectiveRate -> smoothedRate

```

הפלט הוא רשימה מדורגת מלאה, לא top-N קבוע — כל צרכן (פרק 5, 6) גוזר ממנה כמות ואופן שונים.

4.8 קרבה דמוגרפית

מקור עצמאי, אורתוגונלי להתאמת תחום עניין: "מה עבד היטב לאנשים עם אותה דמוגרפיה כמוני (גיל, מגדר, מדינה) — בלי קשר לתחום ענייני". רץ על פני כל הקטלוג, לא רק על מתנות שתויגו בתחומי העניין שהאדם בחר. אין צורך בווקטור מלא או ב-cosine similarity — לא-רלוונטיים לאדם (מגדר/גיל שאינם שלו) פשוט לא נכנסים לחישוב כלל:

```

const DEMOGRAPHIC_WEIGHTS = { age: 1.2, gender: 1.0, country: 1.0 }; // יותר משמעותי גיל

async function getDemographicCloseness(giftId, effectiveAgeBucket, gender, country) {
  const ageAgg = await sumAgeColumnAcrossTagTables(giftId, effectiveAgeBucket); // { shown, liked }
  const genderAgg = await sumGenderColumnAcrossTagTables(giftId, gender); // { shown, liked }
  const countryRow = await db.query(
    `SELECT SUM(shown) shown, SUM(liked) liked FROM gift_country_stats
     WHERE gift_id=$1 AND country=$2`, [giftId, country]
  );
}

```

```

);
const rateAge = smoothedRate(ageAgg.liked, ageAgg.shown, GLOBAL_MEAN, K_PRIOR);
const rateGender = smoothedRate(genderAgg.liked, genderAgg.shown, GLOBAL_MEAN, K_PRIOR);
const rateCountry = smoothedRate(countryRow.liked, countryRow.shown, GLOBAL_MEAN, K_PRIOR);
return DEMOGRAPHIC_WEIGHTS.age * rateAge
    + DEMOGRAPHIC_WEIGHTS.gender * rateGender
    + DEMOGRAPHIC_WEIGHTS.country * rateCountry;
}

```

בודד בלבד על פני כל 12 טבלאות <tag>gift_stats_ (אגרגציה חוצת-תגיות) — לא כל העמודות, רק אלה הרלוונטיות לאדם המבקש.

4.9 סינון שיתופי מבוסס פריטים (Item-Based Collaborative Filtering)

מקור שלישי, אורתוגונלי גם לתחום עניין וגם לדמוגרפיה: "אנשים עם דירוגים דומים לשלך אהבו גם X". item-based (דמיון בין זוגות מתנות) ולא user-based (דמיון בין זוגות משתמשים): הקטלוג גדל לאט וקטן יחסית לבסיס המשתמשים הפוטנציאלי, כך שדמיון-מתנות זול לחשב ונשאר תקף זמן רב יותר — לעומת דמיון-משתמשים, $O(users^2)$ שגדל ללא גבול וטוען חישוב מחדש תדיר. מחושב כ- batch job (למשל יומי), לא בזמן בקשה — עקבי עם העיקרון "ללא תשתית ML כבדה" (1.2).

מקור הדאטה: co-rating מתוך self_gift_suggestions בלבד (" $rating \geq 4$ ") — לא recommendations דירוגי צד שלישי, פחות אמינים לפי 3.2). דמיון מחושב כ- Jaccard על קבוצות המשתמשים שאהבו כל מתנה:

```

-- בקשה בזמן לא, מתוזמן batch job
WITH liked AS (
    SELECT user_id, gift_id FROM self_gift_suggestions WHERE rating >= 4 AND gift_id IS NOT NULL
),
pairs AS (
    SELECT a.gift_id AS gift_a, b.gift_id AS gift_b, COUNT(*) AS both_liked
    FROM liked a JOIN liked b ON a.user_id = b.user_id AND a.gift_id < b.gift_id
    GROUP BY a.gift_id, b.gift_id
    HAVING COUNT(*) >= MIN_COOCCURRENCE_SUPPORT -- מומלצת מחדל ברירת: 5
)
-- similarity = Jaccard: both_liked / |liked(A) U liked(B)|
-- ביותר הגבוה similarity-ה בעלי השכנים (10: מומלצת מחדל ברירת) TOP_K את רק שומרים gift_id לכל

```

בזמן בקשה: שליפה זולה ואינדקסית בלבד — אם לאדם יש מתנה שדירג 4–5, שולפים את שכנותיה מ- gift_neighbors מסננים לפי budget ודודוף, ומוסיפים כמועמדים לרשימה המדורגת (4.7).

5. זרימה — המלצות עצמיות

5.1 Trigger זרימה כללית

- Trigger: POST /api/self-recommendations/generate (ידני כרגע; cron בבעתיד).
- שליפת profileData: user_profiles: interests, negative_prefs, free_text, birth_date, gender, country, religion, bio, מ-.
- כניסה לליבת החישוב המשותפת (פרק 4) — זהה לחלוטין למנגנון לאיש קשר, כולל שרשרת ה- backoff, דמוגרפית ו-CF.
- Gemini מחזיר את מכסתו (2 פריטים, ראו 5.2) → נשמר ב- self_gift_suggestions עם batch_id אחד לכל 15 הפריטים, כולל gift_id רלוונטי.
- GET /api/self-recommendations/:id/rate להצגה; PATCH /api/self-recommendations/:id/rate לדירוג (כולל feedback_reason, פרק 7).

5.2 הרכב ה- 15 batch (פריטים, קבוע)

מקור	כמות	איך נוצר
1. טופ-קטלוג גלובלי	3	דגימה אקראית ללא החזרה מתוך top-10 גלובלי (smoothed global rate)
2. Gemini. הקשר מלא	2	פרומפט מלא (פרק 8): interests, free_text, bio, היסטוריית דירוגים
3. תחומי עניין — exploitation + exploration	4	ראו טבלת משנה מטה

מקור	כמות	איך נוצר
4. כללי — exploitation + exploration	4	אותו פיצול כמו #3, אך מתוך "bucket כללי" ולא תחום עניין ספציפי
5. קרבה דמוגרפית	2	דגימה אקראית מתוך top-5 לפי (4.8) getDemographicCloseness על פני כל הקטלוג

תת-מקור (בתוך #3 ו-#4, 2 פריטים כל אחד)	כמות	איך נוצר
Top-5 אקראי (exploitation)	2	בוחרים תגית (אקראית מתוך תחומי העניין של האדם עבור #3; "כללי" קבוע עבור #4), דוגמים 2 מתוך 5 המתנות המדורגות גבוה ביותר בתגית
חקר: 30% תחתון	1	פריט אקראי מתוך 30% המתנות המדורגות נמוך ביותר בתגית — לא מוחרג, כי אין "מתנה רעה באופן מוחלט", רק טעם נישתי שטרם קיבל תמיכה
חקר: אקראי רחב	1	פריט אקראי מתוך שאר המתנות בתגית (לא ב- top-5 וולא ב- 30% התחתון)

הזרקת "תגית לא נבחרת": בכל אחד מ-#3/#4, קיים סיכוי נמוך (offTagProbability, ברירת מחדל מומלצת 5–10%) שהתגית שממנה דוגמים תוחלף בתגית שהאדם לא סימן — לגילוי שינוי טעם.

```
function buildInterestSlots(userTags, allTagsList, offTagProbability) {
  const zeroTags = userTags.length === 0;
  const chosenTag = zeroTags ? "general"
    : (Math.random() < offTagProbability
      ? randomFrom(allTagsList.filter(t => !userTags.includes(t)))
      : randomFrom(userTags));
  const tagGifts = rankedGiftsForTag(chosenTag);
  const top5 = tagGifts.slice(0, 5);
  const bottom30 = tagGifts.slice(-Math.max(1, Math.ceil(tagGifts.length * 0.3)));
  const middle = tagGifts.slice(5, tagGifts.length - bottom30.length);
  return {
    exploitation: sampleWithoutReplacement(top5.length ? top5 : tagGifts, Math.min(2, top5.length || tagGifts.length)),
    explorationNiche: tagGifts.length ? [randomFrom(bottom30)] : [],
    explorationBroad: middle.length ? [randomFrom(middle)] : (tagGifts.length ? [randomFrom(tagGifts)] : []),
  };
}
```

מקרי קצה: (א) אין לאדם אף תחום עניין נבחר → כל 4 הסלוטים של מקור #3 עוברים ל"כללי" (מקור #4 מקבל בפועל 8 פריטים). (ב) בתגית הנבחרת פחות מ-5 מתנות → exploitation דוגם מכמה שיש, בלי הכפלה. (ג) אין אף מתנה בתגית → שלושת תתי-המקורות ריקים, מוחלפים ב- Gemini הקשר-מלא (מקור #2) לשמירה על 15 פריטים.

6. זרימה — המלצות לאיש קשר

שונה עקרונית מהמנגנון העצמי: מי שמבקש את ההמלצה אינו הנהנה הסופי — האינטרס שלו הוא למצוא את המתנה הטובה ביותר עבור מישהו אחר. לכן הכמות דינמית (עד תקרה), לא קבועה, ולולאת הפידבק קלת-משקל (פרק 7).

16.1 בקשה ראשונה

מקור	כמות	איך נוצר
1. מתנות מוכחות שהוא אוהב	(6–10 דינמי)	getProvenCandidates (תיעדוף למה שהאדם עצמו כבר אישר, ורק אז למה שהוכיח את עצמו לאנשים דומים (backoff, 4.4–4.7))
2. Gemini	(8–12 דינמי, ממלא את מה שנשאר)	geminiCount = 2 + (6 - provenCount)
3. טופ-קטלוג גלובלי	(2 קבוע)	שני הפריטים עם ה-smoothed_rate הגבוה ביותר, אחרי סינון budget ובדיקת דדוף

provenCount + geminiCount + 2 = 10 תמיד עבור העמוד הראשון. proven tiers, קרבה דמוגרפית (4.8) ו- CF מוזנים כולם לאותה רשימה מדורגת אחת (4.7) — אין סדר-קדימות נוקשה ביניהם, רק ציון משותף.

```
function getProvenCandidates(contact, cleanInterests, budget, excludeIds) {
  const tier1 = contact.linked_user_id
```



```

    ? getSelfApprovedGifts(contact.linked_user_id, excludeIds) // rating >= 4 ב-
    self_gift_suggestions
    : [];
    const remaining = 6 - tier1.length;
    const tier2 = remaining > 0
    ? getTopCatalogCandidates(scoredCandidates, remaining, excludeIds) // מתוך 4.7, כולל CF
    : [];
    return [...tier1, ...tier2];
}

```

("tier1 מוכח אישית") זמין רק לאיש קשר מקושר, ונשען על ההיסטוריה של האדם עצמו. ("tier2 מוכח לאנשים דומים") הוא הרשימה המדורגת המאוחדת (4.7) וממלא את מה שחסר עד 6 — כולל את כל המקרים שבהם אין tier1 linked_user_id (תמיד ריק).

- מקור "מתנות מוכחות" מוצג תמיד (as-is) כותרת/תיאור/מחיר מקוריים מהקטלוג, עם רענון (search_query/מחיר) — לא עובר ניסוח מחדש ע"י Gemini, בניגוד למקור Gemini.
- מקור "טופ-קטלוג גלובלי" עובר סינון budget ובדיקת דדופ לפני שהוא נבחר.
- (recommendation_calls (9 מתעדכן פעם אחת בלבד לכל — session עיבוד קריאת ה-Gemini של העמוד הראשון).

6.2 "חפש עוד" — Pagination מבוסס-Compute

Gemini נקרא פעם אחת בלבד לכל (6.1) session בקשת "חפש עוד" נוספת היא compute-based בלבד — ממשיכה לרדת ברשימה המדורגת (4.7) שכבר חושבה, ללא קריאת LLM נוספת וללא עדכון (rate limit (9 :

```

async function searchMore(contactId, batchId) {
  const shownCount = await countRows('recommendations', { batch_id: batchId });
  if (shownCount >= 30) return { done: true, reason: 'CAP_REACHED' };

  const alreadyShownIds = await getShownGiftIds(batchId); // session, ב- שהוצג מה כל
  const excludeSet = new Set([...alreadyShownIds, ...getExclusionSet(contact)]); // 30 דדופ יום

  const nextPage = scoredCandidates // 6.1-ב שחושבה המאוחדת המדורגת מהרשימה
    .filter(c => !excludeSet.has(c.gift_id))
    .slice(0, Math.min(PAGE_SIZE, 30 - shownCount)); // PAGE_SIZE ברירת 10 מומלצת מחדל ברירת

  await saveRecommendations(nextPage, { contact_id: contactId, batch_id: batchId, source: 'compute' });
  return { done: nextPage.length === 0, items: nextPage };
}

```

אם הרשימה המדורגת מתכלה לפני 30 (למשל קטלוג צעיר/תגית נדירה), searchMore מחזיר done=true עם פחות פריטים — אין דרישה להגיע בהכרח לתקרה.

6.3 תקרה ומעקב session

תקרה: 30 פריטים סה"כ לאיש קשר, ברמת ה- (batch_id) session ללא לכל החיים. אחרי שה-session מוצג (30 פריטים, או שהרשימה נגמרה), בקשה עתידית לאותו איש קשר פותחת batch_id חדש עם קריאת Gemini חדשה, כפוף מחדש ל-rate limit.

- — recommendations.batch_id נוצר פעם אחת עם הבקשה הראשונה; משותף לכל הפריטים כולל כל עמודי "חפש עוד".
- 'gemini' — recommendations.source לעמוד הראשון בלבד, 'compute' לכל שאר העמודים — נדרש כדי לספור נכון מול rate limit ולדעת מה לשלף שוב.

7. לולאת הפידבק

7.1 פידבק במנגנון העצמי (1-5 + סיבה)

- → 4-5 rating שאלה, נחשב הצלחה מלאה.
- → 1-3 rating נשאלת שאלת "מה בעיקר לא עבד?" (גם 3, לא רק 1-2 — "בסדר" עדיין אות למידה קריטי).
- " — WRONG_CONCEPT הכיוון בכלל לא מתאים לו".
- " — WRONG_PRODUCT הרעיון סביר, אבל המוצר הספציפי הזה לא".

- " — TOO_GENERIC הכל טכנית בסדר, אבל זה לא מספיק מיוחד/מרגש".

7.2 מיפוי סיבה לפעולה

Rating	Reason	פעולה על הקטלוג	פעולה על פרופיל/contact
4–5	—	global_liked/shown += 1 good_gifts_catalog; UPSERT gift_country_stats <tag> gift_stats אם יש מדינה עבור כל תגית רלוונטית	אין כתיבה ל-gift_history (2.11)
1–3	WRONG_CONCEPT	global_shown += 1 בלבד	category_tag לא (null, negative_prefs (7.6). shown נרשם אך negative_prefs נכתב
1–3	WRONG_PRODUCT	global_shown += 1 בלבד	ה-gift_id נכנס לרשימת דדופ בלבד ; הקטגוריה נשארת פתוחה
1–3	TOO_GENERIC	global_shown += 1 בלבד	לא נוגע ב-negative_prefs

shown נספר אך ורק על המלצות שדורגו בפועל — לא על כל המלצה שהוצגה. אם מתוך N הצעות המשתמש דירג רק 2, רק שתיים אלה מעדכנות shown/liked בקטלוג. זה תקף במלואו למנגנון העצמי בלבד — במנגנון לאיש קשר קיים חריג מכוון, ראו 7.4.

7.3 פידבק במנגנון לאיש קשר (בינארי)

שונה משמעותית מהזרימה העצמית: אין סף rating שמפעיל שאלה נוספת, ואין שלוש אפשרויות סיבה — שני כפתורים בלבד, "מתאים" / "לא מתאים", ללא כל שלב המשך:

- "מתאים" → נשמר rating=5 עדכון הקטלוג זהה למקרה 4–5 הרגיל (7.2): global_liked/shown += 1, gift_stats <tag> הרלוונטיים.
- "לא מתאים" → נשמר rating=2, feedback_reason=null עדכון זהה למקרה "1–3 ללא reason" (7.2):
global_shown += 1 בלבד, ה-gift_id נכנס לדדופ, negative_prefs נכתב.

אין צורך בשדה ENUM ייעודי ל"מתאים/לא מתאים" — ממפים לשני ערכי rating (5 / 2) ומשתמשים בתשתית הקיימת (7.2) כלשונה. השינוי היחיד הוא בשכבת ה-API/UI שלא חושפת כלל את שאר ערכי הדירוג (1, 3, 4) ולא את שאלת הסיבה בזרימה הזו.

7.4 מנגנון "צ'אנס שני" למתנות שלא דורגו

קיים סיכוי גבוה שהמדרג (צד שלישי) יראה הצעה ולא יטרח לדרג אותה כלל. שתיקה ממושכת עדיין צריכה להזין את המערכת במשהו:

שלב	תנאי	פעולה
חשיפה ראשונה	הפריט הוצג (6.1 או 6.2), rating עדיין NULL	רגיל — ממתינים לפידבק, ללא פעולה נוספת
צ'אנס שני	עברו X ימים (ברירת מחדל מומלצת: 3–5), rating עדיין NULL	הפריט חוזר ומוצג שוב לאותו איש קשר — פטור חד-פעמי מהדדופ הרגיל לחשיפה הנוספת הזו בלבד
עדיין ללא תגובה	עברו עוד X ימים מהחשיפה השנייה, rating עדיין NULL	נקבע אוטומטית — rating=2, feedback_reason=null מפורש (7.3), כולל כניסה לדדופ הרגיל כמו "לא מתאים"

זהו חריג מכוון ומוגבל-היקף לעיקרון "shown נספר רק על מה שדורג בפועל" (7.2) — שנשאר תקף במלואו למנגנון העצמי. כאן הרציונל הפוך: מי שמדרג עושה זאת עבור מישהו אחר, ולכן ציפייה לדירוג מלא על כל הצעה אינה ריאלית. שני חלוניות הזדמנות לפני שהעדר-תגובה הופך לאיתות שלילי הם פשרה בין איסוף דאטה לבין לא "להעניש" מתנה טובה שסתם לא זכתה לתשומת לב.

7.5 יעוד הכתיבה של פידבק: לוקאלי אצל המדרג, לא אצל האדם המדורג

כל פידבק (negative_prefs, feedback_reason) שנשמר מדיירוג שיוסי נותן על המלצה עבור דני, נכתב אך ורק ל-user_profiles.negative_prefs (לעולם לא ל-contacts.negative_prefs הרשומה המקומית של יוסי על דני) — לעולם לא ל-user_profiles.negative_prefs של דני עצמו, גם אם דני מקושר. דירוג של יוסי הוא הערכת צד שלישי, לא אמירה אמיתית של דני על עצמו. user_profiles.negative_prefs של דני מתעדכן אך ורק כתוצאה מפידבק ישיר שדני עצמו נותן, דרך המנגנון העצמי שלו (פרק 5).

7.6 category_tag גישור בין category ו-negative_prefs

הבעיה קיימת בפועל רק עבור המלצות Gemini טהורות (gift_id הוא null). המלצה קטלוגית כבר יש תגית מוכרת: primaryTag שחושב ב-4.4, או התגית הדומיננטית של הפריט (עבור מקורות שלא בהכרח עברו התאמה לפי cleanInterests, כמו tier1 טופ-גלובלי).

```
function resolveCategoryTag(rec, matchedGiftId, primaryTag) {
    if (matchedGiftId) {
        return primaryTag ?? getDominantTag(matchedGiftId);
    }
    return isValidVocabTag(rec.category_tag) ? rec.category_tag : null; // Gemini טהור
}
```

עבור Gemini טהור, מבקשים באותה קריאה שדה נוסף — category_tag — ערך יחיד מה- Master Tag List (2.7), או null אם שום תגית לא מתאימה. parseAndValidate מוודא תקינות מול הרשימה בדיוק כמו עבור tags; אם לא תקין — נשמר null, נכפה מיפוי גרוע.

8. בניית הפרומפט וקריאה ל-Gemini

דת / מדינה / מגדר / גיל / שם: היעד פרטי
cleanInterests (תגיות): עניין תחומי
free_text: ניואנסים
budget_min-budget_max: תקציב

(עד 4) promptNegatives (!לחלוטין הימנע) קשיחות שליליות הנחיות

dedup list (תחזור אל) כבר שהוצעו מתנות

[רלוונטי אם]

(להשראה או לשלב שקול) מתאימות שנמצאו מהקטלוג מוכחות מתנות:

- promptCatalogExamples[0]
- promptCatalogExamples[1]

(title, description, estimated_price, category, category_tag, search_query) ההמלצות עם JSON: מבוקש פלט

```
const response = await callGemini(prompt);
const recommendations = parseAndValidate(response); // JSON schema + tags נרמול
category_tag
const matchedGiftId = matchToPromptCatalogExamples(recommendations, promptCatalogExamples);
await saveRecommendations(recommendations, {
    user_id, contact_id, event_id,
    gift_id: matchedGiftId,
    category_tag: resolveCategoryTag(recommendations, matchedGiftId, primaryTag),
    batch_id, source: 'gemini',
});
```

parseAndValidate כולל נרמול תגיות מול ה- Master Tag List כדי ש-Gemini לא "ימציא" תגיות שיפגעו בעתיד ב-tags cleanInterests. matchToPromptCatalogExamples מקשר gift_id וגם Gemini אימץ אחת מהדוגמאות שהוצעו לו cas-is, מכעט כדי שפידבק עתידי יזין לאותה שורת קטלוג ולא ייצור כפילות.

9. Rate Limiting

- Per-user: 5 קריאות ל-Gemini לשבוע (recommendation_calls).
 - Global: 40 קריאות ל-Gemini ליום (api_usage).
 - במנגנון לאיש קשר, ה-rate limit נחל ונספר רק על הבקשה הראשונה של כל session (source='gemini', 6.1) — קריאות "חפש עוד" (source='compute', 6.2) ואינן נספרות.
- שקול rate limit וקל (למשל per-minute) ע"פ "חפש עוד" עצמו, כדי למנוע שימוש לרעה (בקשות אוטומטיות) על שכבת compute שאינה כפופה למגבלת Gemini.

10. Pre-processing — נתוני זרע

בהקמת המערכת: כ-300 מתנות זרע. קיטלוג ראשוני (למשל מרשימת רעיונות ידנית/מאתר) עובר תיוג אוטומטי ע"י LLM מול ה-Master Tag List הסגורה (2.7) — כולל ריבוי תגיות לפריט בודד כשרלוונטי (למשל "תרמוס טיולים" מתויג גם טיולים/טבע וגם ספורט).

- הפריטים נכנסים עם `axis_seed=true` ללא `global_shown/liked` מומצאים — `shown=0/liked=0` הטבלאות הרלוונטיות (`good_gifts_catalog`) `gift_stats_<tag>` המטרה היא מוכנות מבנית (`tags, price, search_query`) מוכנים), לא הזרקת סטטיסטיקה מזויפת — `smoothedRate` דואג לכך שפריט עם `shown=0` יתחיל בדיוק ב- `GLOBAL_MEAN`.
- ברגע שיש 300 פריטי `seed` מתווגים על פני כל 11 התגיות + כללי, גם משתמשים מוקדמים ביותר מקבלים מיד מאגר סביר לכל אחד ממקורות #4/#3 בהרכב ה- `batch` העצמי (5.2), לא רק ל-1# (טופ-גלובלי).
- `K_PRIOR` כדאי גבוה יותר (15–20) בשלב מוקדם — מושך חזק יותר כלפי הממוצע כשיש מעט, `shown` במיוחד ברמת ה- `pair`.
- `GLOBAL_MEAN` עצמו לא אמין עם מעט דאטה כולל — מומלץ לקבע ידנית בקונפיג (0.6) עד ~500+ `ratings` מצטברים, ואז לעבור לחישוב דינמי.

11. Edge Cases

- **Cold Start:** עם ה- `backoff` ההיררכי, בשלב מוקדם כמעט תמיד `pairCandidates` לא יעברו `MIN_PAIR_SUPPORT`, והמערכת תיפול ל- `interest` וזו `global` התנהגות מתוכננת — אין להנמיך ספים מלאכותית.
- סתירה פנימית בקלט: נפתר ב-3.2 — `Negative` תמיד גובר על `Positive` באותה רמת מקור סמכות; בין שתי רמות שונות, הרמה האמינה יותר (`self` גוברת תמיד).
- **Negative Prefs:** מגבלת כמות קשיחה — רק 4 האחרונים נכנסים לפרומפט (4.1). שדרוג עתידי: `TTL/Decay` לתגיות שליליות ישנות.
- **Echo Chamber:** נפתר ע"י הרכב ה- `batch` המעורב (5.2, 6.1) — חלק קבוע מכל `batch` הוא `exploration` טהור ו/או `Gemini` יצירתי.
- מתנת נישא (`Niche Popularity Pollution`) נפתר ע"י שרשרת ה- (4.5) `backoff` בכל רמותיה — מתנה לא זוכה ל- `rate` מספיקי לרמה עד שיש לה מספיק תצפיות בדיוק באותה רמה.
- **Tag Drift** - `Gemini` נפתר ע"י נרמול לרשימת `vocabulary` סגורה בזמן שמירת המלצה חדשה (פרק 8).
- כפילויות בקטלוג: לפני `INSERT` ל- `good_gifts_catalog`, נדרשת בדיקת דמיון טקסטואלי בסיסי (`title/search_query`) קיימת כפילות — מעדכנים את הסטטיסטיקה של השורה הקיימת ולא פותחים שורה חדשה. אלגוריתם הדמיון המדויק (`exact match`) מנומרל (`Levenshtein / embeddings`) טרם נבחר — ראו נספח א'.
- **Link/Price Rot:** `last_verified_at` בקטלוג + הנחיה ל- `Gemini` ליצור `search_query` עדכני בכל הצעה מחדש. רצף דירוגים נמוכים על מתנה קיימת → מסירים אותה או מורידים משקלה.
- **Dedup** לאותו נמען: `recommendations.created_at` על חלון 30 יום לפי `contact_id; getExclusionSet` מאחד את זה עם `selfAlreadyShown` כל `gift_id` שכבר הוצג לנמען במנגנון העצמי שלו, ללא הגבלת זמן — מוזרק גם ל- `getProvenCandidates` לרשימת "אל תחזור" בפרומפט.

12. תכונות שנדחו במפורש (לא ב-MVP)

" `gift_history` **קנייתי בפועל** ": כפתור/פעולה נפרדת לסימון רכישה בפועל, לא קשורה למנגנון הדירוג. דירוג 4–5 הוא איתות ה-"הצלחה" היחיד שקיים ב- MVP — "אהבתי את הרעיון" לבין "קנייתי בפועל". שום מנגנון פעיל במסמך זה תלוי בה.

acrossSiblingContacts: כיסוי דו-כיווני גם על פני כל ה- `contacts` שמצביעים לאותו (`linked_user_id`) למשל שני אנשים שונים יצרו את אותו דוד כאיש קשר) — ייבנה יחד עם `gift_history`. עד אז, שני מבקשים שונים עלולים תיאורטית לקבל, בלי ידיעה, את אותה "מתנה מוכחת" עבור אותו נמען — פשרה מקובלת ב-MVP.

צ'אנס שני למנגנון העצמי: מנגנון דומה ל-7.4 (עבור המנגנון לאיש קשר) לא הורחב למנגנון העצמי — שם ההנחה היא שהמשתמש מדרג עבור עצמו, כך שהרציונל (צד שלישי, פחות מוטיבציה לדרג) אינו קיים.

13. רשימת API Endpoints מרומזת

רשימה מסונתזת מכלל הזרימות שתוארו במסמך — לא תועדה במפורש בגרסאות קודמות, מרוכזת כאן לנוחות המימוש:

Endpoint	שיטה	תיאור
<code>/api/self-recommendations/generate</code>	POST	מפעיל <code>batch</code> חדש של 15 המלצות עצמיות (5.1–5.2)
<code>/api/self-recommendations</code>	GET	שליפת ה- <code>batch</code> האחרון עבור המשתמש (לפי <code>batch_id</code>)
<code>/api/self-recommendations/:id/rate</code>	PATCH	דירוג 1–5 + <code>feedback_reason</code> אופציונלי (7.1–7.2)
<code>/api/recommendations</code>	POST	בקשה ראשונה עבור — <code>contact_id/event_id</code> כולל קריאת <code>Gemini</code> (6.1)

Endpoint	שיטה	תיאור
/api/recommendations/search-more	POST	עמוד נוסף עבור batch_id — compute-only, Gemini (6.2)
/api/recommendations/:id/rate	PATCH	פידבק בינארי: FIT / NOT_FIT בלבד (7.3), ממופה ל-5/2 rating=
/api/hosted-events	POST	יצירת אירוע מאורח + קהל יעד (14.1, 14.3)
/api/groups	POST	יצירת קבוצה (14.2)
/api/groups/:id/invite	POST	הזמנה ישירה של contact (14.2)
/api/groups/:id/invite-link	POST	יצירת/רענון invite_links מסוג GROUP (14.2, 2.17)
/api/contact-invite-link	POST	יצירת/רענון invite_links מסוג CONTACT_LIST (14.2, 2.17)
/api/join/:token	POST	לחיצה על קישור הזמנה — מפעיל את הלוגיקה ב-14.2
/api/group-invites/:id/respond	PATCH	קבלה/דחייה של הזמנה שדורשת אישור (ACCEPT/DECLINE)

14. שיתוף אירועים, קבוצות ורשימות תפוצה

תוספת עצמאית יחסית לשאר המסמך — אינה נוגעת במנגנון ההמלצות/הניקוד עצמו (פרקים 4–9), אלא בשכבה שקובעת מי רואה אירוע שמשמש בוחר לחשוף. המטרה המעשית: לאפשר למשתמש שמארח אירוע ("החתונה שלי") לשתף אותו בבת אחת עם קבוצה שלמה (למשל "המשפחה"), במקום להוסיף כל איש קשר בנפרד.

14.1 hosted_events מול — events שני מושגים שונים

events (2.3) היא תזכורת פרטית: משתמש עוקב אחרי תאריך של איש קשר שלו ("ליום ההולדת של דוד"), ורק הוא רואה את זה. (2.14) hosted_events היא ההפך בכיוון: משתמש הוא הנחגג, ובוחר באופן אקטיבי למי לחשוף את זה, כדי שאנשים אחרים יזכרו לחשוב עליו כשמגיע הזמן. שתי הטבלאות אינן מקושרות זו לזו ואינן חולקות סכימה — מדובר בזרימות שונות לגמרי שרק במקרה עוסקות שתיהן ב"תאריך חשוב".

14.2 חברות בקבוצה — הזמנה ישירה מול קישור

שני נתיבים בלבד להצטרפות לקבוצה, שניהם דורשים חשבון רשום — אין דרך לצרף מישהו שאין לו חשבון ב-Giftly.

נתיב	מי יכול	מה קורה
DIRECT_INVITE	יוצר הקבוצה בוחר מתוך אנשי הקשר המקושרים (linked) של בלבד	אם ל-user_id המוזמן require_approval_for_group_invites=false (מחדל) → group_members.status='MEMBER' אם true → status='INVITED' (ממתין לתגובת המוזמן) (POST /api/group-invites/:id/respond)
LINK	כל מי שברשותו invite_links.token מסוג GROUP (2.17) — לא חייב להיות איש קשר של היוצר מראש	אם הלוחץ מחובר ורשום → group_members.status='MEMBER' מיידי, → joined_via='LINK', אם אינו רשום — מופנה להרשמה; הקבוצה אינה רלוונטית לו עד שיירשם

אין קשר סכימתי בין group_members ל- (2.16) contacts. הצטרפות לקבוצה אינה יוצרת רשומת contact אוטומטית. במקום זאת, ברגע שקבוצה חבר חדש הופך ל-MEMBER (בכל נתיב) ויוצר הקבוצה עדיין אין לו contact מקושר לאותו user_id, ששכבת ה-UI מציגה הצעה להוסיפה — לא אכיפה, לא כתיבה אוטומטית לטבלה. קישור אישי לרשימת התפוצה (target_type='CONTACT_LIST', 2.17) invite_links, עובד באותו עיקרון בדיוק, רק שהתוצאה שונה: לחיצה ע"י משתמש רשום יוצרת עבור owner_user_id רשומת contacts חדשה (עם linked_user_id=הלוחץ), אם עדיין אין לו כזו. זהו נתיב חד-כיווני — הלוחץ נוסף כ-contact של בעל הקישור, אין הנחה שגם ההפך קורה.

14.3 פתרון קהל היעד של אירוע

(2.18) event_audience מכיל אוסף שורות-מטרה עבור hosted_event; הקהל בפועל הוא האיחוד (Union) של כולן, לא בחירה יחידה. פונקציית הפתרון, מנקודת המבט של "אילו אירועים גלויים למשתמש X":

```
async function getVisibleHostedEvents(userId) {
  const viaAllContacts = await db.query(`
    SELECT he.* FROM hosted_events he
    JOIN event_audience ea ON ea.hosted_event_id = he.id AND ea.target_type = 'ALL_CONTACTS'
    JOIN contacts c ON c.user_id = he.owner_user_id AND c.linked_user_id = $1
  `, [userId]);
```

```

const viaContact = await db.query(`
  SELECT he.* FROM hosted_events he
  JOIN event_audience ea ON ea.hosted_event_id = he.id AND ea.target_type = 'CONTACT'
  JOIN contacts c ON c.id = ea.target_contact_id AND c.linked_user_id = $1
`, [userId]);

const viaGroup = await db.query(`
  SELECT he.* FROM hosted_events he
  JOIN event_audience ea ON ea.hosted_event_id = he.id AND ea.target_type = 'GROUP'
  JOIN group_members gm ON gm.group_id = ea.target_group_id
  AND gm.user_id = $1 AND gm.status = 'MEMBER'
`, [userId]);

return dedupeById([...viaAllContacts, ...viaContact, ...viaGroup]);
}

```

שלושת המקורות (ALL_CONTACTS, CONTACT, GROUP) נשלפים בנפרד ומאוחדים; אין קדימות ביניהם, בדיוק כמו שקהל היעד עצמו הוא איחוד (14, פתיח). דדופ פשוט לפי id — אדם אותו אירוע נגיש למשתמש דרך יותר מנתיב אחד (למשל גם חבר בקבוצה וגם contact ישיר), הוא מופיע פעם אחת בלבד.

14.4 כלל הנגישות (Reachability)

כלל אחד שחוזר בכל המנגנון: אדם יכול להיות מטרה (של קבוצה, או של שיתוף אירוע ישיר) רק אם הוא בר-הישג בתוך האפליקציה — כלומר יש `contacts.linked_user_id` (או, במקרה של קבוצה, `group_members.user_id`), `event_audience` ב-CONTACT — משתמש רשום מטבעו). `contact` לא-מקושר אינו זמין לבחירה כמטרה. `CONTACT` — `event_audience` — זמין להזמנה לקבוצה — אין דרך "לשתף" עם מישהו שאין לו חשבון להיכנס אליו ולראות. זהה בעקרונו לכלל שכבר קיים במסמך 8.11 בגרסה הקודמת, — `best-available-signal` — כפי שסינון קשיח בזמן בחירת מטרה, לא רק ניקוד.

נספח א' — קבועים לכיול וסיכום החלטות מוצר פתוחות

כל הקבועים במסמך מסומנים עם ברירת מחדל קונקרטית, כדי שהמימוש הראשוני לא ייתקע — כולם ניתנים לכיול בהמשך ללא שינוי מבני.

קבוע	ברירת מחדל מומלצת	היכן משמש
MIN_PAIR_SUPPORT	20	4.5 — סף תמיכה לצירוף זוגי (<code>interest+age / interest+country</code>)
MIN_TAG_SUPPORT	15	4.5 — סף תמיכה לרמת תחום עניין בודד
GLOBAL_MEAN	0.6 (קבוע ידנית עד ~500 ratings, + דינמי)	4.6 — נקודת ברירת המחדל ל- <code>smoothing</code>
(K_PRIOR tag)	10	4.6
(K_PRIOR pair)	15	4.6 — רועש יותר, נמשך חזק יותר לממוצע
AGE_BUCKETS	18-24 / 25-34 / 35-44 / 45-54 / 55-64 / 65+	4.4 — לא נבדק אמפירית
DEMOGRAPHIC_WEIGHTS	<code>age=1.2, gender=1.0, country=1.0</code>	4.8
MIN_COOCCURRENCE_SUPPORT	5	4.9 — <code>CF</code> <code>co-rating</code>
(TOP_K שכני CF)	10	4.9 — כמות שכנים נשמרת ל- <code>gift_neighbors</code>
offTagProbability	5–10%	5.2 — הסתברות הזרקת תגית לא-נבחרת
(PAGE_SIZE חפץ עוד)	10	6.2
תקרת (session contact)	30	6.3
X ימים (צ'אנס שני)	3–5 ימים לכל חלון (6–10 סה"כ)	7.4
Rate limit — per-user	5 קריאות Gemini/שבוע	9
Rate limit — global	40 קריאות Gemini/יום	9
Seed data — כמות	300~פריטים	10
Coarse filter LIMIT	100	4.2

החלטות מוצר שטרם התקבלו (לא טכניות — דורשות קלט אנושי לפני שהמימוש שלהן סופי; עד אז ניתן לממש עם הכיוון המסתמן בסוגריים):

- אלגוריתם דמיון טקסטואלי לדדופ קטלוג — exact match מנורמל / Levenshtein / embeddings מומלץ להתחיל ב- exact match מנורמל, הכי פשוט למימוש).
- קריטריון "אימץ כמעט" — matchToPromptCatalogExamples — אחד עם אותו אלגוריתם דדופ שנבחר למעלה.
- שדה recommendations.score עבור המלצות exploration טהורות ללא match לקטלוג — מומלץ null.
- האם צריך rate limit — מומלץ להתחיל בלי, ולהוסיף רק אם ייצפה שימוש לרעה בפועל.
- cold start — האם היחסים בין 5 המקורות צריכים להתאים דינמית לבשלות הקטלוג, בדומה למנגנון לאיש קשר (provenCount)? — נתוני הזרע (פרק 10) כבר נותנים כיסוי סביר מההתחלה.
- קישור רשימת התפוצה האישי (14.2), (target_type='CONTACT_LIST' מוגדר כחד-כיווני (הלוחץ נוסף כ- contact של בעל הקישור בלבד) — הונח כברירת מחדל תואמת למודל ה- contacts הקיים; אם נדרשת התנהגות חדדית (שני הצדדים נוספים זה כ- contact של זה), זהו שינוי קטן ומבודד ב-14.2.